Generating Language Bindings using C++ Reflection

CALLUM PIPER

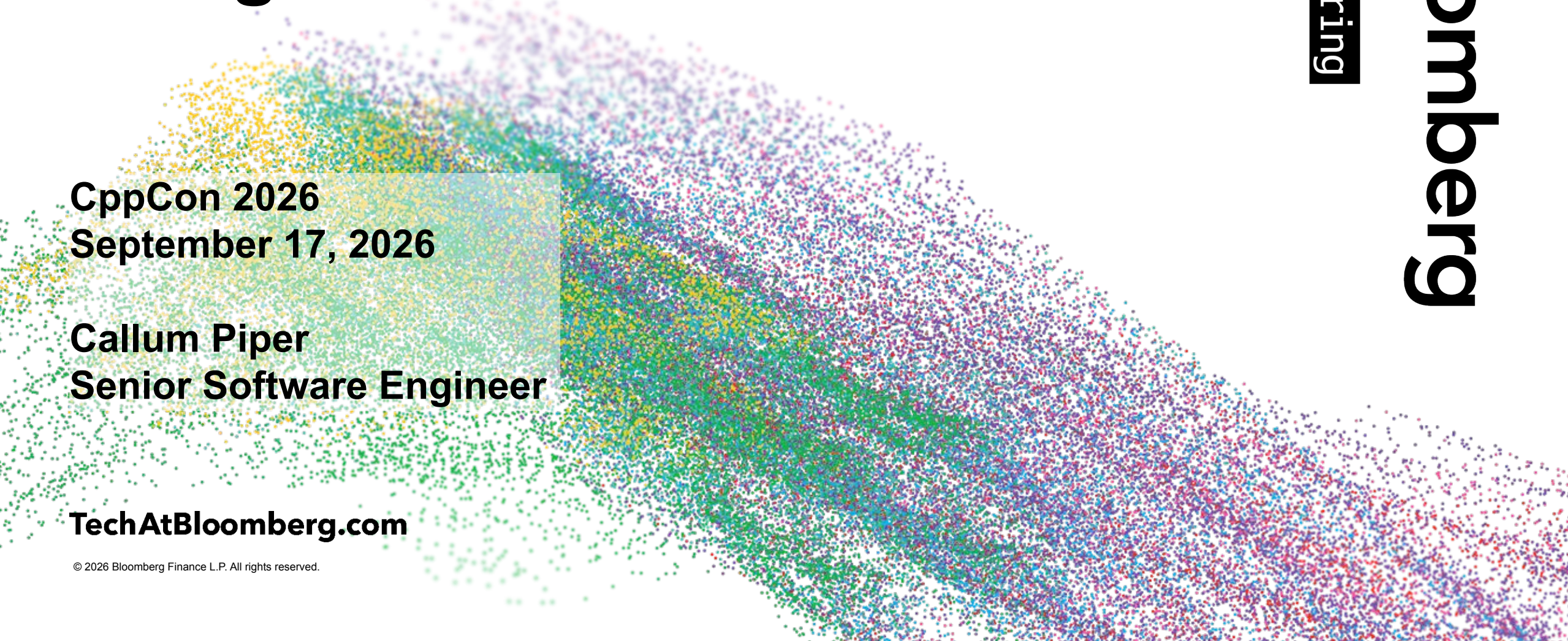




Generating Language Bindings using C++ Reflection

Bloomberg

Engineering



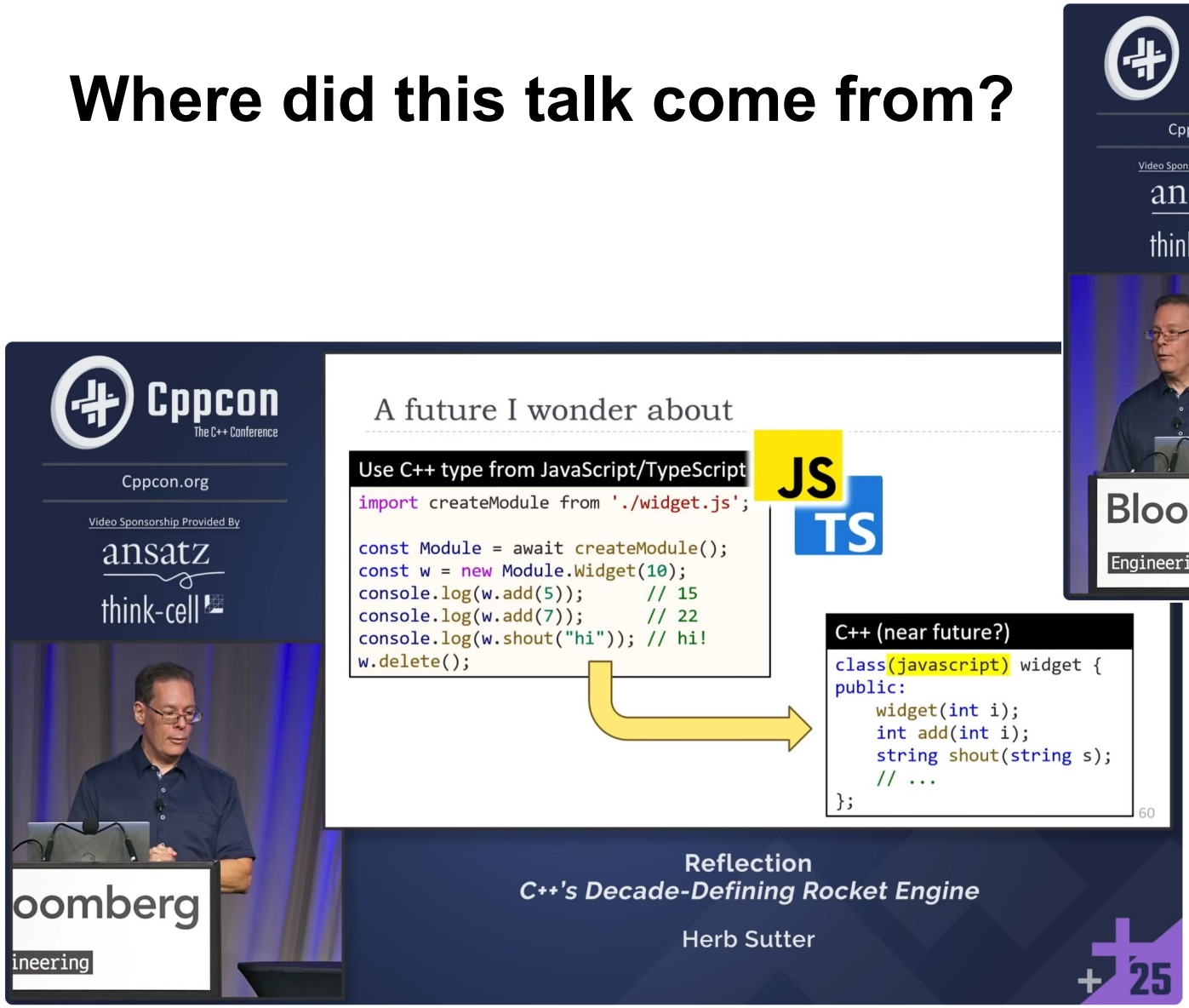
CppCon 2026
September 17, 2026

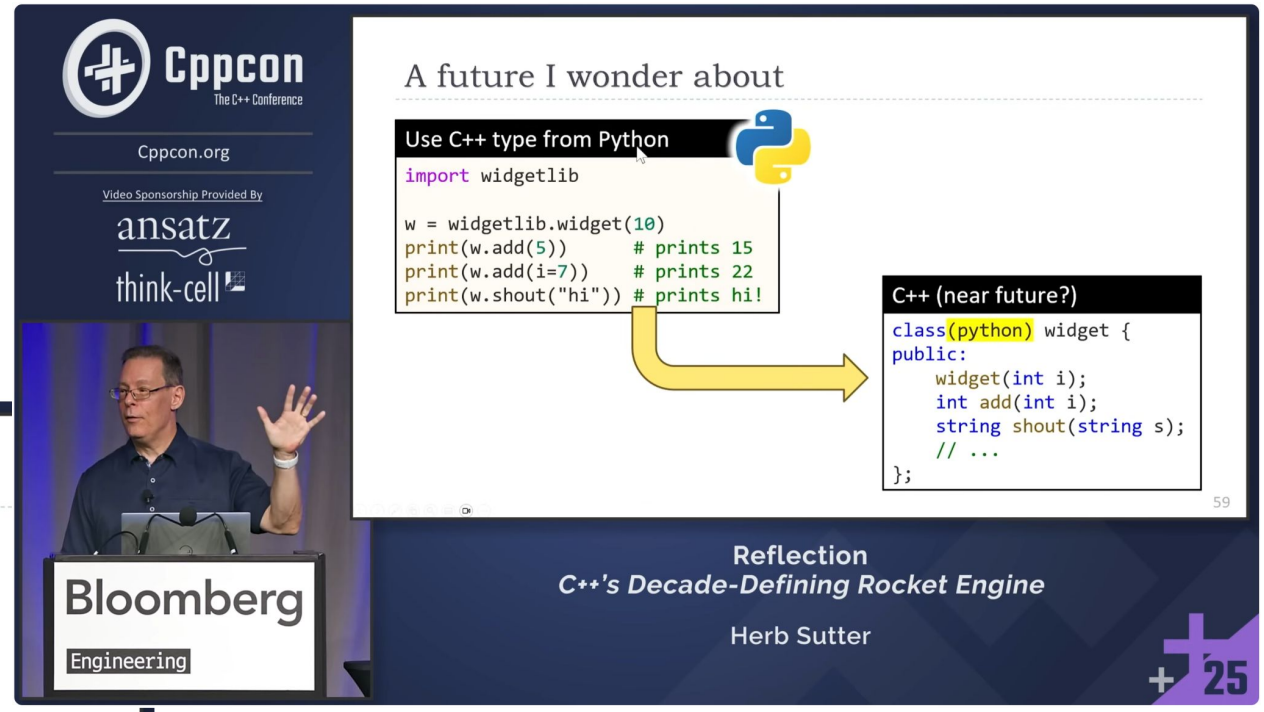
Callum Piper
Senior Software Engineer

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Where did this talk come from?





https://youtu.be/7z9NNrRDHQU

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Forget C++29…What can we do today?

CMakeLists.txt

```
add_library(simplelib STATIC
    widget.cpp
    anotherclass.cpp
)

generate_python_bindings(simplelib widget.h api::Widget)
generate_js_bindings(simplelib widget.h api::Widget)
```

Library to link to

Header file to include

Type to generate bindings for

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering

Starting from…

Standard project

```
.
├── LICENSE
├── README.md
├── CMakeLists.txt
└── simplelib
    ├── CMakeLists.txt
    ├── anotherclass.cpp
    ├── anotherclass.h
    ├── widget.cpp
    └── widget.h
```

```
add_library(simplelib STATIC
    widget.cpp
    anotherclass.cpp
)

generate_python_bindings(simplelib widget.h api::Widget)
generate_js_bindings(simplelib widget.h api::Widget)
```

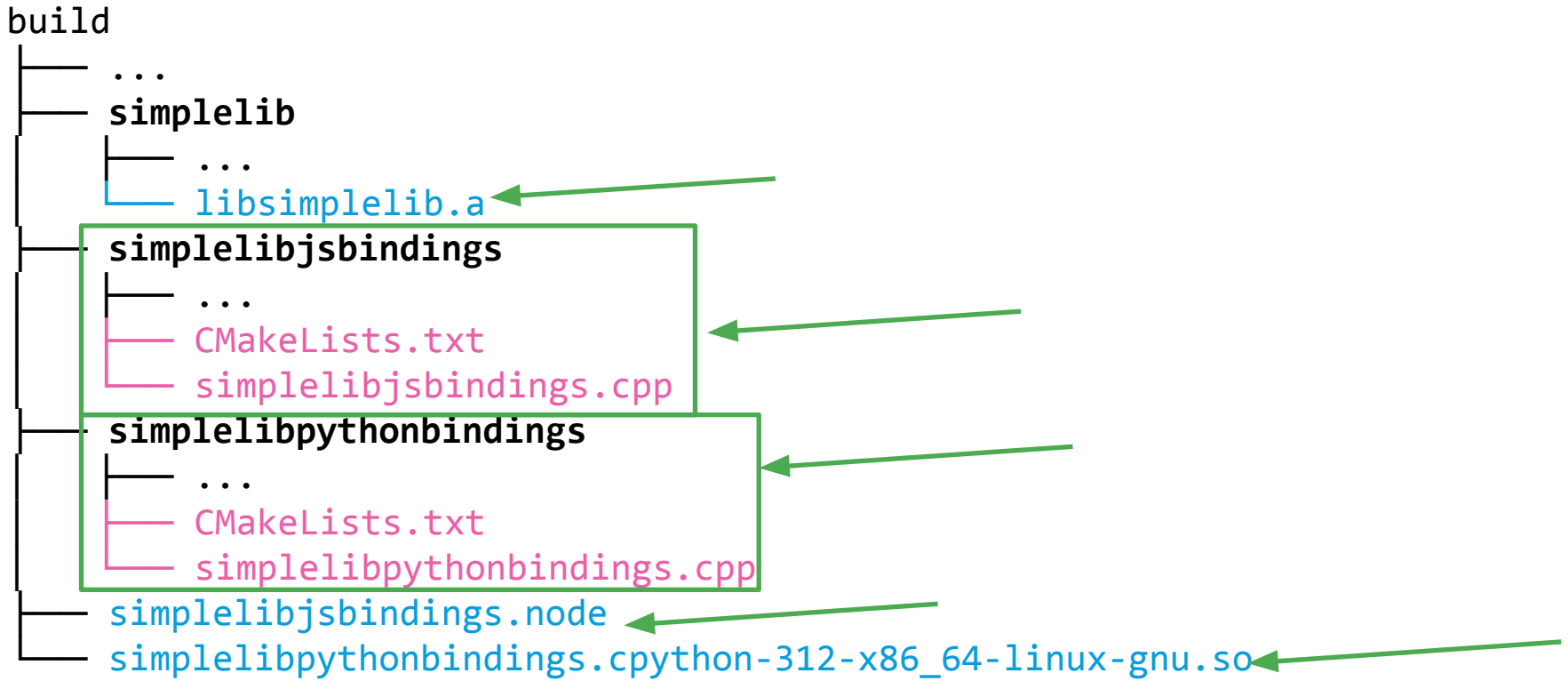

© 2026 Bloomberg Finance L.P. All rights reserved.



What does this generate?

Creates separate CMake targets and outputs for:

- The C++ library and language bindings



© 2026 Bloomberg Finance L.P. All rights reserved.

So how does this work?

Combination of:

- CMake code generation from templates (.in)
- Reflection
- Binding boilerplate
  - For Python, pybind11 (or similar) takes care if this
  - For JavaScript/TypeScript, I've used MirrorBridge by Francisco Thiesen
    - https://github.com/FranciscoThiesen/mirror_bridge/tree/main

We will look at both of the first two parts, and assume the binding "just works"

© 2026 Bloomberg Finance L.P. All rights reserved.



Let’s look at the C++ first

simplelibpythonbindings.cpp

```cpp
#include "widget.h"

#include <pythonbindings.h>

PYBIND11_MODULE(simplelibpythonbindings, m)
{
    pythonbindings::bindClass<^^api::Widget>(m);
}
```

simplelibjsbindings.cpp

```cpp
#include "widget.h"

#include <jsbindings.h>

JS_MODULE(simplelibjsbindings, m)
{
    jsbindings::bindClass<^^api::Widget>(env, m);
}
```


© 2026 Bloomberg Finance L.P. All rights reserved.

Let’s look at the Python version…

```
1  template <meta::info CppClass>
2  void bindClass(py::module& module)
3  {
4      // auto pyClass = py::class_<api::Widget>(module, "Widget", py::module_local());
5      auto pyClass = py::class_<typename [:CppClass:]>(
6          module, meta::identifier_of(CppClass).data(), py::module_local());
7
8      bindConstructors(pyClass);
9      bindMemberFunctionsWithArgs(pyClass);
10     bindEnums(pyClass);
11 }
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering

Quick Digression - meta::substitute()

Given template function:

```
template <typename T, typename U, meta::info Refl> void fn(T i, U s, Refl r);
```

This statement:

```
meta::info refl = ...
meta::substitute(^^fn, {^^int, ^^std::string, ^^refl})
```

Returns:

```
^^fn<int, std::string, refl>
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering

Quick Digression - meta::substitute()

Given template function:

```
template <typename T, typename U, meta::info Refl> void fn(T i, U s, Refl r);
```

This statement:

```
meta::info refl = ...
meta::substitute(^^fn, {^^int, ^^std::string, meta::reflect_constant(refl)})
```

Returns:

```
^^fn<int, std::string, refl>
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Binding Constructors

```
1 template <class CppClass> void bindConstructors(py::class_<CppClass>& pyClass) {
2     template for (constexpr meta::info fn : getPyInitFns<CppClass>())     {
3         // add constructor bindings Widget::Widget(std::string) -> pyClass.def(py::init<std::string>());
4         pyClass.def([:fn:]());
5     }
6 }
```

```
1 template <typename CppClass> consteval auto getPyInitFns() {
2     auto pyInitProto = getPyInit(); // find py::init<>() template function
3
4     std::vector<meta::info> pyInitFns;
5
6     for (meta::info cns : publicConstructorsOf(^^CppClass))   {
7         auto cnsTypeParams = meta::parameters_of(cns) | views::transform(meta::type_of);
8         pyInitFns.push_back(meta::substitute(pyInitProto, cnsTypeParams)); // populate template params of py::init<>()
9     }
10    return define_static_array(pyInitFns);
11 }
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Binding Constructors

```
1 template <class CppClass> void bindConstructors(py::class_<CppClass>& pyClass) {
2     template for (constexpr meta::info fn : getPyInitFns<CppClass>())     {
3         // add constructor bindings Widget::Widget(std::string) -> pyClass.def(py::init<std::string>());
4         pyClass.def([:fn:]());
5     }
6 }
```

Run-time

```
1 template <typename CppClass> consteval auto getPyInitFns() {
2     auto pyInitProto = getPyInit(); // find py::init<>() template function
3
4     std::vector<meta::info> pyInitFns;
5
6     for (meta::info cns : publicConstructorsOf(^^CppClass))   {
7         auto cnsTypeParams = meta::parameters_of(cns) | views::transform(meta::type_of);
8         pyInitFns.push_back(meta::substitute(pyInitProto, cnsTypeParams)); // populate template params of py::init<>()
9     }
10    return define_static_array(pyInitFns);
11 }
```

Compile-time

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Finding Constructors

```
1 constexpr auto current_ctx = meta::access_context::current();
2
3 consteval auto publicConstructorsOf(meta::info classInfo)
4 {
5     return meta::members_of(classInfo, current_ctx)
6             | views::filter(meta::is_user_declared)
7             | views::filter(meta::is_constructor);
8 }
```



Binding member functions

```
1 template <typename CppClass>
2 void bindMemberFunctionsWithArgs(py::class_<CppClass>& pyClass) {
3     template for (constexpr meta::info fn : publicMemberFunctionsOf(^^CppClass)) {
4         constexpr meta::info bindMemFnWithArgs = instantiateBindMemFnWithArgs<CppClass>(fn);
5         [:bindMemFnWithArgs:](pyClass);
6     }
7 }
```

Run-time

```
1 template <typename CppClass, meta::info Fn, meta::info... ArgNames>
2 void bindMemFnWithArgs(py::class_<CppClass>& pyClass) {
3     constexpr auto fnName = meta::identifier_of(Fn);
4     // pyClass.def("divide", &Widget::divide, py::arg("numerator"), py::arg("denominator"));
5     pyClass.def(fnName.data(), &[:Fn:], py::arg([:ArgNames:])...);
6 }
```

Run-time



Binding member functions (cont.)

```
1  // reflection of bindMemFnWithArgs<CppClass, meta::info, meta::info, meta::info>
2  //                                                fn    , arg1 name , arg2 name
3  template <typename CppClass>
4  consteval meta::info instantiateBindMemberFnWithArgs(meta::info fn) {
5      std::vector<meta::info> templateParams;
6      templateParams.push_back(^^CppClass);
7      templateParams.push_back(meta::reflect_constant(fn));
8
9      for (meta::info arg : meta::parameters_of(fn)) {
10         templateParams.push_back(
11             meta::reflect_constant(meta::reflect_constant_string(meta::identifier_of(arg))));
12     }
13
14     return meta::substitute(^^bindMemFnWithArgs, templateParams);
15 }
```

Compile-time

Finding Member functions

```
1  constexpr auto current_ctx = meta::access_context::current();
2
3  consteval auto publicMemberFunctionsOf(meta::info classInfo)
4  {
5      return define_static_array(meta::members_of(classInfo, current_ctx)
6                                 | views::filter(meta::is_user_declared)
7                                 | views::filter(meta::is_function)
8                                 | views::filter(std::not_fn(meta::is_constructor))
9                                 | views::filter(std::not_fn(meta::is_destructor)));
10 }
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

C++ Reflection Summary

Starting from:

```
1 #include "widget.h"
2
3 #include <pythonbindings.h>
4
5 PYBIND11_MODULE(simplelibpythonbindings, m)
6 {
7     pythonbindings::bindClass<^^api::Widget>(m);
8 }
```

We can generate all the bindings for the class

This works along the same lines for JavaScript

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Jupyter Notebook

Generate | + Code | + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ...

```
from simplelibpythonbindings import Widget
w = Widget()
w_int = Widget(5)
w_str = Widget("Hello")

w.testFunction()

w.divideValues(6, 2)
```

[3] ✓ 0.0s

```
5
Hello
This is a test

3
```

```
w.divideValues?
```

[4] ✓ 0.0s

```
Docstring: divideValues(self: simplelibpythonbindings.Widget, numerator: int, denominator: int) -> int
Type:      method
```

Node.js

```
Welcome to Node.js v18.19.1.
Type ".help" for more information.
> const simplelib = require('./simplelibjsbindings.node');
undefined
> const widget = new simplelib.Widget();
undefined
> const widget_int = new simplelib.Widget(5);
5
undefined
> const widget_str = new simplelib.Widget("Hello");
Hello
undefined
> widget.testFunction();
This is a test
undefined
> widget.divideValues(6, 2);
3
```



CMake Generation

For each language binding, we need two files:

1. CMakeLists.txt
   - This creates the target - the language binding library

2. *.cpp
   - This creates the bindings for a class which we reflect on

© 2026 Bloomberg Finance L.P. All rights reserved.



generate_python_bindings() function

```
1  function(generate_python_bindings BINDING_TARGET BINDING_ENTRY_HEADER BINDING_ENTRY_TYPE)
2      set(PYTHON_BINDING_DIR ${CMAKE_BINARY_DIR}/${BINDING_TARGET}pythonbindings)
3      file(MAKE_DIRECTORY ${PYTHON_BINDING_DIR})
4
5      configure_file(bindingutils/pythonCMakeLists.txt.in
6                     ${PYTHON_BINDING_DIR}/CMakeLists.txt)
7
8      configure_file(bindingutils/pythonbindings.cpp.in
9                     ${PYTHON_BINDING_DIR}/${BINDING_TARGET}pythonbindings.cpp)
10
11     add_subdirectory(${PYTHON_BINDING_DIR} ${PYTHON_BINDING_DIR}) # source_dir binary_dir
12 endfunction()
```



pythonCMakeLists.txt.in

```
pybind11_add_module(@BINDING_TARGETNAME@pythonbindings
    @BINDING_TARGETNAME@pythonbindings.cpp
)

target_include_directories(@BINDING_TARGETNAME@pythonbindings PRIVATE
    ${CMAKE_SOURCE_DIR}/@BINDING_TARGETNAME@
    bindingutils
)

target_link_libraries(@BINDING_TARGETNAME@pythonbindings PRIVATE
    @BINDING_TARGETNAME@
)

set_target_properties(@BINDING_TARGETNAME@pythonbindings PROPERTIES
    LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR})
install(TARGETS @BINDING_CLASSNAME@pythonbindings LIBRARY DESTINATION .)
```


© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering



pythonCMakeLists.txt.in

```
1  pybind11_add_module(@BINDING_TARGETNAME@pythonbindings
2      @BINDING_TARGETNAME@pythonbindings.cpp
3  )
4
5  target_include_directories(@BINDING_TARGETNAME@pythonbindings PRIVATE
6      ${CMAKE_SOURCE_DIR}/@BINDING_TARGETNAME@
7      bindingutils
8  )
9
10 target_link_libraries(@BINDING_TARGETNAME@pythonbindings PRIVATE
11     @BINDING_TARGETNAME@
12 )
13
14 set_target_properties(@BINDING_TARGETNAME@pythonbindings PROPERTIES
15     LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR})
16 install(TARGETS @BINDING_CLASSNAME@pythonbindings LIBRARY DESTINATION .)
```


© 2026 Bloomberg Finance L.P. All rights reserved.



pythonCMakeLists.txt.in

```
1  pybind11_add_module(@BINDING_TARGETNAME@pythonbindings
2      @BINDING_TARGETNAME@pythonbindings.cpp
3  )
4
5  target_include_directories(@BINDING_TARGETNAME@pythonbindings PRIVATE
6      ${CMAKE_SOURCE_DIR}/@BINDING_TARGETNAME@
7      bindingutils
8  )
9
10 target_link_libraries(@BINDING_TARGETNAME@pythonbindings PRIVATE
11     @BINDING_TARGETNAME@
12 )
13
14 set_target_properties(@BINDING_TARGETNAME@pythonbindings PROPERTIES
15     LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR})
16 install(TARGETS @BINDING_CLASSNAME@pythonbindings LIBRARY DESTINATION .)
```


© 2026 Bloomberg Finance L.P. All rights reserved.



pythonCMakeLists.txt.in

```
pybind11_add_module(@BINDING_TARGETNAME@pythonbindings
    @BINDING_TARGETNAME@pythonbindings.cpp
)

target_include_directories(@BINDING_TARGETNAME@pythonbindings PRIVATE
    ${CMAKE_SOURCE_DIR}/@BINDING_TARGETNAME@
    bindingutils
)

target_link_libraries(@BINDING_TARGETNAME@pythonbindings PRIVATE
    @BINDING_TARGETNAME@
)

set_target_properties(@BINDING_TARGETNAME@pythonbindings PROPERTIES
    LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR})
install(TARGETS @BINDING_CLASSNAME@pythonbindings LIBRARY DESTINATION .)
```


© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering



pythonCMakeLists.txt.in

```
pybind11_add_module(@BINDING_TARGETNAME@pythonbindings
    @BINDING_TARGETNAME@pythonbindings.cpp
)

target_include_directories(@BINDING_TARGETNAME@pythonbindings PRIVATE
    ${CMAKE_SOURCE_DIR}/@BINDING_TARGETNAME@
    bindingutils
)

target_link_libraries(@BINDING_TARGETNAME@pythonbindings PRIVATE
    @BINDING_TARGETNAME@
)

set_target_properties(@BINDING_TARGETNAME@pythonbindings PROPERTIES
    LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR})
install(TARGETS @BINDING_CLASSNAME@pythonbindings LIBRARY DESTINATION .)
```


© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering

Generated CMakeLists.txt for Python

```
pybind11_add_module(simplelibpythonbindings
    simplelibpythonbindings.cpp
)

target_include_directories(simplelibpythonbindings PRIVATE
    ${CMAKE_SOURCE_DIR}/simplelib
    bindingutils
)

target_link_libraries(simplelibpythonbindings PRIVATE
    simplelib
)

set_target_properties(simplelibpythonbindings PROPERTIES
    LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}
)
install(TARGETS simplelibpythonbindings LIBRARY DESTINATION .)
```


© 2026 Bloomberg Finance L.P. All rights reserved.



Generated CMakeLists.txt for JavaScript

```
1  add_library(simplelibjsbindings SHARED
2      simplelibjsbindings.cpp
3  )
4  target_include_directories(simplelibjsbindings PRIVATE
5      ${CMAKE_SOURCE_DIR}/simplelib
6      bindingutils
7      /usr/include/node
8  )
9  target_link_libraries(simplelibjsbindings PRIVATE
10     simplelib
11 )
12 set_target_properties(simplelibjsbindings PROPERTIES
13     LIBRARY_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}
14     PREFIX ""
15     SUFFIX ".node"
16 )
17 install(TARGETS simplelibjsbindings LIBRARY DESTINATION .)
```



pythonbindings.cpp.in

```
1 #include <@BINDING_HEADERNAME@>
2
3 #include <pythonbindings.h>
4
5 PYBIND11_MODULE(@BINDING_TARGETNAME@pythonbindings, m)
6 {
7     pythonbindings::bindClass<^^@BINDING_ENTRY@>(m);
8 }
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering

pythonbindings.cpp.in

```
1 #include <@BINDING_HEADERNAME@>
2
3 #include <pythonbindings.h>
4
5 PYBIND11_MODULE(@BINDING_TARGETNAME@pythonbindings, m)
6 {
7     pythonbindings::bindClass<^^@BINDING_ENTRY@>(m);
8 }
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering

pythonbindings.cpp.in

```
1 #include <@BINDING_HEADERNAME@>
2
3 #include <pythonbindings.h>
4
5 PYBIND11_MODULE(@BINDING_TARGETNAME@pythonbindings, m)
6 {
7     pythonbindings::bindClass<^^@BINDING_ENTRY@>(m);
8 }
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering

pythonbindings.cpp.in

```
#include <@BINDING_HEADERNAME@>

#include <pythonbindings.h>

PYBIND11_MODULE(@BINDING_TARGETNAME@pythonbindings, m)
{
    pythonbindings::bindClass<^^@BINDING_ENTRY@>(m);
}
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering



pythonbindings.cpp.in

```
#include <@BINDING_HEADERNAME@>

#include <pythonbindings.h>

PYBIND11_MODULE(@BINDING_TARGETNAME@pythonbindings, m)
{
    pythonbindings::bindClass<^^@BINDING_ENTRY@>(m);
}
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering



Generated pythonbindings.cpp

```cpp
#include "widget.h"

#include <pythonbindings.h>

PYBIND11_MODULE(simplelibpythonbindings, m)
{
    pythonbindings::bindClass<^^api::Widget>(m);
}
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering

What about Rust bindings?

- I couldn’t see how to do this
  - Problem is how to generate C-like wrapper functions to call C++
    - Same for bindgen or cxx - as far as I can tell
- Reflection can’t generate functions (yet!)
  - Only generates aggregate types (struct of data members)
- Rust Bindings can’t bind to template functions (instantiated or not)
- Happy to be corrected on any of these points

© 2026 Bloomberg Finance L.P. All rights reserved.



Going back to the start

CMakeLists.txt

```
1 add_library(simplelib STATIC
2     widget.cpp
3     anotherclass.cpp
4 )
5
6 generate_python_bindings(simplelib widget.h api::Widget)
7 generate_js_bindings(simplelib widget.h api::Widget)
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering

Going back to the start

```
1 add_library_with_bindings(simplelib STATIC
2     widget.cpp PYTHON JAVASCRIPT api::Widget
3     anotherclass.cpp
4 )
```

```
1 add_library_with_bindings(simplelib STATIC
2     widget.cpp PYTHON api
3     foo.cpp PYTHON api
4     anotherclass.cpp
5 )
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering



Opting in or out

What if a class or namespace isn’t precise enough?

- Annotations is one option
- Or custom solutions

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering



Annotations

```
1  struct [[=jsbinding::exclude]] Widget {
2
3      [[=pythonbinding::exclude]]
4      enum class ClassOptions {
5          NONE,
6          SOME,
7          ALL,
8      };
9
10     Widget() = default;
11     Widget(int i);
12     Widget(std::string s);
13
14     ~Widget() = default;
15
16     void testFunction();
17     int divideValues(int numerator, int denominator);
18
19 [[=pythonbinding::exclude]] void unsupportedFunction();
20 };
```



Annotations

```
1  struct [[=jsbinding::exclude]] Widget {
2
3      [[=pythonbinding::exclude]]
4      enum class ClassOptions {
5          NONE,
6          SOME,
7          ALL,
8      };
9
10     Widget() = default;
11     Widget(int i);
12     Widget(std::string s);
13
14     ~Widget() = default;
15
16     void testFunction();
17     int divideValues(int numerator, int denominator);
18
19 [[=pythonbinding::exclude]] void unsupportedFunction();
20 };
```

But I really don't like this



Alternative

```
#include "widget.h"

#include <pythonbindings.h>

PYBIND11_MODULE(simplelibpythonbindings, m)
{
    pythonbindings::bindClass<^^api::Widget>(m);
}
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering

Alternative

```
#include "widget.h"

#include <pythonbindings.h>

PYBIND11_MODULE(simplelibpythonbindings, m)
{
    pythonbindings::bindClass<^^api::Widget,
                              pythonbindings::exclude<"enum", "*unsupported*">>(m);
}
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

Alternative

```
#include "widget.h"

#include <pythonbindings.h>

PYBIND11_MODULE(simplelibpythonbindings, m)
{
    pythonbindings::bind<pythonbindings::include<"api">,
                         pythonbindings::exclude<"enum", "*unsupported*">>(m);
}
```

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.



Bloomberg

Engineering

Alternative

```cpp
#include "widget.h"

#include <pythonbindings.h>

PYBIND11_MODULE(simplelibpythonbindings, m)
{
    pythonbindings::bind<pythonbindings::include<"api">,
                         pythonbindings::exclude<"enum", "*unsupported*">>(m);
}
```

```
add_library_with_bindings(simplelib STATIC
    widget.cpp PYTHON JAVASCRIPT include:api exclude:enum, *unsupported*
    anotherclass.cpp
)
```


© 2026 Bloomberg Finance L.P. All rights reserved.



A step further…

```
{
    "python": {
        "parseHeaders": [
            "widget.h",
            "foo.h"
        ],
        "include": [
            {"namespace": "api"}
        ],
        "exclude": [
            {"type": "enum"},
            {"pattern": "*unsupported*"}
        ]
    },
    "javascript": {
      ...
    }
}
```

```
add_library_with_bindings(simplelib STATIC
    widget.cpp
    anotherclass.cpp
)
```

1. CMake parses JSON config
2. Generates C++ from template using config
3. Compiles generated C++ using reflection to bind selected types

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Bloomberg

Engineering

A step further…

```
{
    "python": {
        ...
    },
    "javascript": {
        "parseHeaders": [
            "widget.h"
        ],
        "include": [
            {"pattern": "Widget"}
        ],
        "exclude": [
            {"pattern": "*unsupported*"}
        ]
    }
}
```

```
add_library_with_bindings(simplelib STATIC
    widget.cpp
    anotherclass.cpp
)
```

1. CMake parses JSON config
2. Generates C++ from template using config
3. Compiles generated C++ using reflection to bind selected types

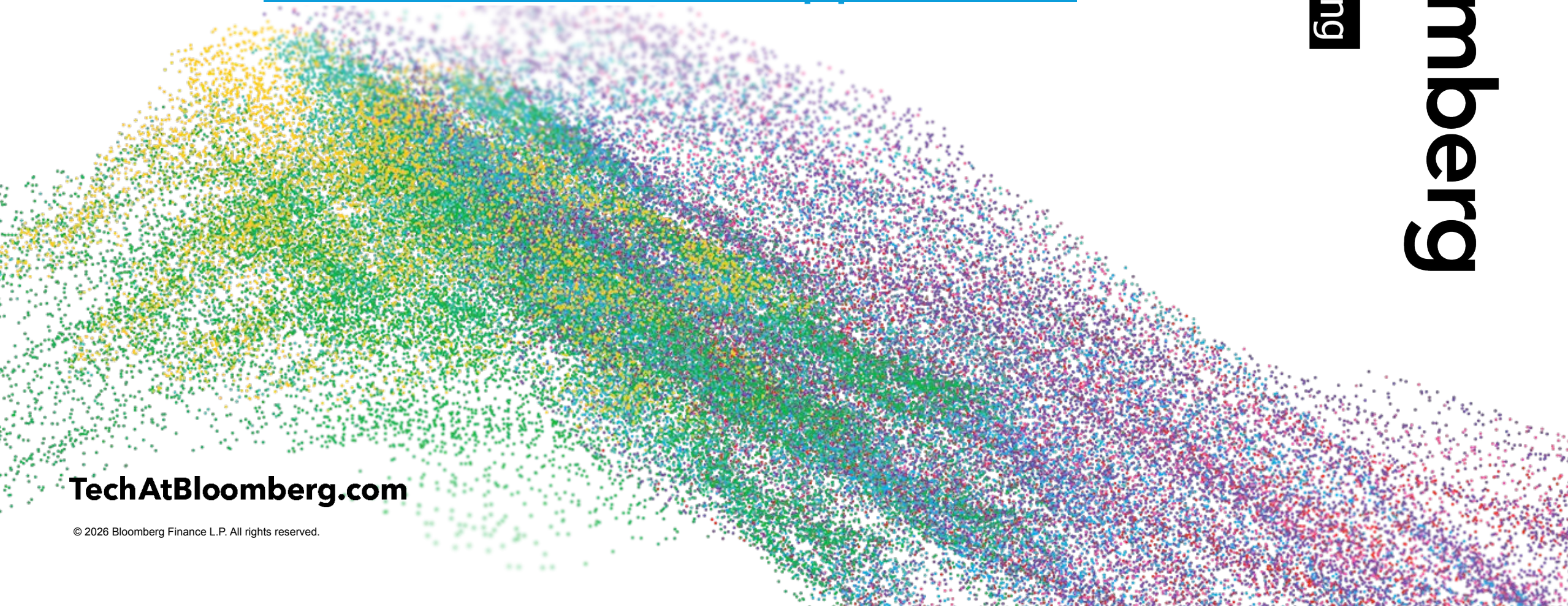
© 2026 Bloomberg Finance L.P. All rights reserved.

Thank you!

LinkedIn: www.linkedin.com/in/callum-piper-3691373

Bloomberg

Engineering



TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.

Questions?

Bloomberg

Engineering

Interested in C++ @ Bloomberg?
We are hiring: bloomberg.com/engineering

Career Opportunities

Learn more about our work

TechAtBloomberg.com

© 2026 Bloomberg Finance L.P. All rights reserved.